

NAME: Adrias, Angelica S.

SUBJECT: 6ADET

PROJECT PROPOSAL

APP NAME: Code Royal

THE PROBLEM:

I struggle to stay motivated when practicing programming through notes and online quizzes, so I want to turn coding practice into a game where answering programming questions feels like battling enemies and progressing through levels.

WHO IS THIS FOR:

Audience:

- Myself
- Classmates who struggle finding motivation on learning

What they do today:

- Read lecture notes
- Practice coding using simple exercises

What Code Royal does:

- Makes coding entertaining like a game
- Giving feedback through the HP loss, XP gain, and level progression

CORE FEATURES:

#	Feature	MVP Status	Flutter Pieces it needs	Honest estimate
1	Battle Quiz	KEEP	TextField, ElevatedButton, StatefulWidget, setState()	8 hours
2	HP Battle System	KEEP	LinearProgressIndicator, Container, Row, Column, state updates	6 hours
3	XP and Level System	KEEP	Model class, setState(), conditional logic	5 hours
4	Progress Saving	KEEP	shared_preferences, loading and saving player data	6 hours
5	Random Programming Questions	KEEP	Question model, List, Random, filtering by difficulty	7 hours
6	Player Profile	KEEP	Navigator.push, Card, ListTile, reusable widgets	4 hours

STRETCH GOALS:

1. Different enemy types, by the end when they faced the boss, they do a code battle
2. If possible animated attack effects and battle backgrounds
3. Badges for each milestone
4. Sound effects and battle music
5. Attack animation

HOW MY APP SAVES DATA:

Should two users see the same data?

- No, since each player have their own progress stored on their own device because it is designed as a single player learning app.

Estimated Data Volume:

- 1 player profile
- 20 – 40 completed quiz battles
- Around 30-40 stored progress values like XP, unlocked levels, and scores.

Storage Choice:

- shared_preferences

Why and Tradeoff?

- I chose shared_preferences because Code Royal only needs to save one player progress locally on their device. It is simple to implement and matches the app's current scope without requiring accounts, internet access, or a backend server. The progress will only exist on the device where the game is played. It will not automatically sync between multiple devices like Firebase or Supabase.

What exactly gets saved?

CLASS	FIELDS SAVED	STORAGE KEY
PlayerProgress	playerName, level, xp, totalScore	player_progress
UnlockedLevels	completedLevels, unlockedLevels	player_levels
Settings	selectedDifficulty	game_settings

Have I tried it yet?

- Not exactly but I have watched a few videos of the process, I plan to fully try it during the coding itself. I love to test and edit mid way to learn.

ONE THING I WANT TO ADD THAT THE COURSE DID NOT TEACH:

Feature: *Battle Sound Effects and Background Music*

Flutter Package: audioplayers

Does it run on web: *Yes, but browser audio has some autoplay restrictions. The app will still function without audio if the browser blocks it.*

Backup Plan: *The web version will use silent gameplay if audio cannot play automatically. During the final demonstration, I will record the mobile version where audio works normally.*

Core or Stretch: *Stretch Goal*

HOW MY PROJECT RUNS WHEN SOMEONE ELSE OPENS IT:

Keep device_preview?	No. It is useful only while developing
Runs in browser?	Not yet. The goal is that every screen works using flutter run -d web-server before the final submission.
Hardware fallback?	Yes. Audio is optional and the game works even without it.

Public GitHub repository?	Yes. The repository will contain only source code and sample game data.
API keys or passwords?	None currently required. No personal user data or API keys will be stored in the repository.

DATA THE APPS NEED TO REMEMBER:

THING	FIELDS	WHERE SAVED
Player Progress	Name, Level, XP, Total Score	shared_preferences
Completed Levels	List of finished battles	shared_preferences
Unlocked Levels	Highest unlocked difficulty or stage	shared_preferences
Game Settings	Difficulty preference	shared_preferences
Programming Questions	Question text, correct answer, difficulty	Stored locally inside the app as model data.

SCREENS:

SCREEN	PURPOSE	FLUTTER EVIDENCE
Main Menu	Start battle, continue progress, open profile.	Scaffold, Card, ElevatedButton
Battle Screen	Programming battle with HP bars and answer input.	StatefulWidget, TextField, LinearProgressIndicator
Result Screen	Victory or defeat summary with XP earned.	Navigator.push, Column, Card
Player Profile Screen	Display player level, XP, score, and completed levels.	ListView.builder, reusable cards

RISKS:

Battle State: Making the battle system feel fun while correctly updating HP, XP, and victory conditions.

Is it still a risk? Yes, but it is now smaller because Module 5 introduced StatefulWidget and setState() for updating the interface.

First Step: Create one enemy with one programming question and verify HP changes correctly.

Saving Player Progress: Saving and loading player progress correctly after closing the application.

First Step: Build a small prototype using shared_preferences that saves only player level and XP before adding scores and unlocked levels.

WHAT CHANGED AND WHY:

SECTION	PRELIM	PROPOSAL 2	WHY IT CHANGED
Audience	Mostly for myself and anyone who lacks motivation.	Myself and classmates learning programming who want interactive practice.	Professor feedback said the audience was too vague.
Core Features	Five features without Flutter implementation details.	Each feature now includes Flutter widgets and realistic development hours.	Modules 4 and 5 taught layouts, state, input, lists, and navigation.
Progress Saving	Progress is saved automatically.	Progress is stored locally using shared_preferences.	Module 5 introduced stateful apps, and the revision requires a storage decision.
Data Plan	Questions listed as remembered data.	Questions remain app content; only player progress is saved.	Simplifies storage and matches the chosen backend.
Stretch Goals	Enemy types, animations, badges.	Added sound effects as another stretch goal.	Recognized additional features outside the course scope.
Risks	One general battle-system risk.	Battle state management plus persistent storage with deadlines.	Module 5 showed that state and persistence are separate implementation challenges.
Project Scope	Game could save progress anytime.	Progress saves after completed battles instead of resuming unfinished battles.	Reduced scope to make the project achievable within the remaining semester.